AI PROJECT

Mahmoud Hazem 2200098

Ahmed Badawy 2200091

Mohamed Abolyazeed 2200337

Abdelraouf Dahy 2102317

INRODUCTION

This project explores the application of Artificial Intelligence (AI) techniques to solve classic two-player games, specifically focusing on [**Specify the chosen game(s): e.g., Tic-Tac-Toe or Sudoku**]. These games, while seemingly simple, offer a valuable platform for understanding core AI concepts such as:

- **Game Theory:** Analyzing game strategies, decision-making under uncertainty, and predicting opponent moves.
- **Search Algorithms:** Developing efficient methods to explore the vast space of possible game states.
- **Heuristic Functions:** Evaluating game positions and guiding search algorithms towards optimal solutions.

By implementing AI algorithms to play these games, we aim to:

- **Demonstrate understanding of fundamental AI principles.**
- **Develop practical problem-solving and programming skills.**
- **Explore the limitations and challenges of AI in game-playing scenarios.**

This project will involve:

- **Game Implementation:** Creating a functional game environment with a user interface.
- **AI Algorithm Development:** Designing and implementing AI algorithms, such as:
    - **Minimax:** A classic game tree search algorithm.
    - **Alpha-Beta Pruning:** An optimization technique for Minimax.
    - **Monte Carlo Tree Search (MCTS):** A probabilistic search algorithm.
- **Algorithm Evaluation:** Testing and comparing the performance of different AI algorithms.
- **Documentation:** Creating comprehensive documentation of the project, including code, design decisions, and results.

Through this project, we will gain valuable insights into the power and limitations of AI in solving challenging problems, while also creating engaging and interactive game experiences.

TIC TAC TOE:

Tic-Tac-Toe is a classic two-player game played on a 3x3 grid. Players take turns marking spaces in the grid with their respective symbols (typically "X" and "O"). The objective is to be the first player to form a line of three of their symbols horizontally, vertically, or diagonally. If all spaces on the grid are filled and no player has achieved a winning line, the game ends in a draw.

Game Logic

- **Board Representation:** The game board can be represented using a 2D array (e.g., a list of lists in Python).

- **Player Turns:** Players alternate turns, marking an empty space on the board.

- **Move Validation:** Each move must be valid, ensuring that the chosen space is within the grid boundaries and is currently empty.

- **Win Condition:** After each move, the game checks for a winning condition:
    - Check all rows for three consecutive symbols.
    - Check all columns for three consecutive symbols.
    - Check both diagonals for three consecutive symbols.
- **Draw Condition:** If the board is full and no player has won, the game ends in a draw.

AI Implementation (Minimax Algorithm)

The Minimax algorithm is a common approach for AI players in two-player games. Here's a simplified explanation:

1. **Game Tree:** The algorithm constructs a game tree representing all possible moves and their consequences.
2. **Evaluation Function:** A heuristic function evaluates the "score" of each board position. For example, a simple heuristic could assign scores based on the number of potential winning lines for each player.
3. **Minimax Search:** The algorithm explores the game tree recursively, assigning scores to each node.
    - **Maximizing Player (AI):** The AI player aims to maximize the score.
    - **Minimizing Player (Human):** The human player aims to minimize the score.
4. **Best Move Selection:** The AI player chooses the move that leads to the highest score according to the Minimax evaluation.

```python
class TicTacToeLogic:
    def __init__(self, player_symbol, opponent_symbol):
        self.board = [['_' for _ in range(3)] for _ in range(3)]
        self.player = player_symbol
        self.opponent = opponent_symbol

    def is_moves_left(self):
        return any('_' in row for row in self.board)

    def evaluate(self):
        lines = self.board + list(zip(*self.board))
        diagonals = [[self.board[i][i] for i in range(3)], [self.board[i][2 - i] for i in range(3)]]
        lines += diagonals

        for line in lines:
            if line.count(line[0]) == 3 and line[0] != '_':
                return 10 if line[0] == self.player else -10
        return 0
```

---

1. Class Definition:

```python
def __init__(self, player_symbol, opponent_symbol):
```

- Initializes the game with the player's symbol (e.g., 'X') and the opponent's symbol (e.g., 'O').
- board: Represents the 3x3 game board using a 2D list.

```python
def is_moves_left(self):
```

- Checks if there are any empty spaces ('_') remaining on the board.
- Returns True if any empty spaces exist, otherwise False.

```python
def evaluate(self):
```

- Evaluates the current state of the board to determine the winner or draw.
- **Lines:** Combines rows and columns of the board into a single list.
- **Diagonals:** Creates a list of both diagonals.
- Iterates through all lines (rows, columns, diagonals):
  - Checks if all elements in a line are the same and not empty:
    - Returns 10 if the line consists of the AI player's symbol.
    - Returns -10 if the line consists of the opponent's symbol.
- Returns 0 if no winner is found.

```python
def minimax(self, depth, is_maximizing_player):
    score = self.evaluate()

    if score == 10 or score == -10:
        return score
    if not self.is_moves_left():
        return 0

    if is_maximizing_player:
        best = -1000
        for i in range(3):
            for j in range(3):
                if self.board[i][j] == '_':
                    self.board[i][j] = self.player
                    best = max(best, self.minimax(depth + 1, False))
                    self.board[i][j] = '_'
        return best
    else:
        best = 1000
        for i in range(3):
            for j in range(3):
                if self.board[i][j] == '_':
                    self.board[i][j] = self.opponent
                    best = min(best, self.minimax(depth + 1, True))
                    self.board[i][j] = '_'
        return best
```

```python
def minimax(self, depth, is_maximizing_player):
```

- Implements the Minimax algorithm for game tree search.
- **depth**: Represents the current depth of the search tree.
- `is_maximizing_player:` Indicates whether it's the AI's turn (maximizing score) or the opponent's turn (minimizing score).
- **Base Cases:**
  - If a win or loss is found (score is 10 or -10), return the score.
  - If the game is a draw (no moves left), return 0.
- **Maximizing Player (AI):**
  - Initializes best to -1000 (worst possible score for the AI).
  - Iterates through all empty cells:
    - Makes a move (places the AI's symbol).

        - Recursively calls minimax() for the opponent's turn.
        - Undoes the move.
        - Updates best to the maximum of the current best and the returned score.
    - Returns the best score.
- **Minimizing Player (Opponent):**
    - Similar to the maximizing player, but initializes best to 1000 and uses min() to find the minimum score.

```python
def find_best_move(self):
    best_val = -1000
    best_move = (-1, -1)

    for i in range(3):
        for j in range(3):
            if self.board[i][j] == '_':
                self.board[i][j] = self.player
                move_val = self.minimax(0, False)
                self.board[i][j] = '_'
                if move_val > best_val:
                    best_move = (i, j)
                    best_val = move_val
    return best_move
```

```python
def find_best_move(self):
```

- Finds the best possible move for the AI player.
- Initializes best_val to -1000 and best_move to an invalid move.
- Iterates through all empty cells:
    - Makes a move.
    - Calls minimax() to evaluate the move.
    - Undoes the move.

    - Updates best_move and best_val if a better score is found.
- Returns the coordinates of the best move.

Sudoku:

Sudoku is a number-placement puzzle played on a 9x9 grid subdivided into nine 3x3 subgrids called "regions". The objective is to fill the grid with digits from 1 to 9 so that:

- **Each row contains all digits from 1 to 9.**
- **Each column contains all digits from 1 to 9.**
- **Each 3x3 region contains all digits from 1 to 9.**

The puzzle starts with some cells already filled with digits, providing clues to solve the remaining empty cells.

Game Logic

- **Board Representation:** The Sudoku board can be represented using a 2D array (e.g., a list of lists in Python).
- **Initial State:** The initial state of the board contains some pre-filled cells with digits and empty cells represented by zeros or empty strings.
- **Constraint Checking:**
    - **Row Check:** Verify that each row contains all digits from 1 to 9 without duplicates.
    - **Column Check:** Verify that each column contains all digits from 1 to 9 without duplicates.
    - **Region Check:** Verify that each 3x3 region contains all digits from 1 to 9 without duplicates.
- **Solving Techniques:**
    - **Backtracking:** A recursive algorithm that explores different possibilities by trying to place digits in empty cells. If a placement leads to a violation of the Sudoku rules, the algorithm backtracks and tries a different placement.
    - **Constraint Propagation:** Techniques like Hidden Single, Naked Single, and Hidden Pair can be used to eliminate possible candidates for certain cells, reducing the search space.

```python
def print_sudoku(board):
    for i in range(len(board)):
        if i % 3 == 0 and i != 0:
            print("- - - - - - - - - - - - - - ")

        for j in range(len(board[0])):
            if j % 3 == 0 and j != 0:
                print(" | ", end="")

            if j == 8:
                print(board[i][j])
            else:
                print(str(board[i][j]) + " ", end="")


def find_next_empty_cell(board):

    for i in range(len(board)):
        for j in range(len(board[0])):
            if board[i][j] == 0:
                return (i, j)

    return None


ex_board = [
    [7, 8, 0, 4, 0, 0, 1, 2, 0],
    [6, 0, 0, 0, 7, 5, 0, 0, 9],
    [0, 0, 0, 6, 0, 1, 0, 7, 8],
    [0, 0, 7, 0, 4, 0, 2, 6, 0],
    [0, 0, 1, 0, 5, 0, 9, 3, 0],
    [9, 0, 4, 0, 6, 0, 0, 0, 5],
    [0, 7, 0, 3, 0, 0, 0, 1, 2],
    [1, 2, 0, 0, 0, 7, 4, 0, 0],
    [0, 4, 9, 2, 0, 6, 0, 0, 7]
]
```

```python
def print_sudoku(board):
```

- This function prints the Sudoku grid in a visually appealing format with grid lines.
- It iterates through each row and column of the board:
    - Prints a horizontal line every three rows.
    - Prints a vertical line every three columns.
    - Prints the current cell value.

```python
def find_next_empty_cell(board):
```

- This function searches the board for the next empty cell (represented by 0).
- It iterates through each row and column of the board:
    - If an empty cell is found, it returns the coordinates (row, column) of that cell.
- If no empty cells are found, it returns None.

```python
def valid_board(board, num, pos):
            return False

    for i in range(len(board)):
        if board[i][pos[1]] == num and pos[0] != i:
            return False

    box_x = pos[1] // 3
    box_y = pos[0] // 3

    for i in range(box_y * 3, box_y * 3 + 3):
        for j in range(box_x * 3, box_x * 3 + 3):
            if board[i][j] == num and (i, j) != pos:
                return False

    return True

def solve_sudoku(board):
    find = find_next_empty_cell(board)
    if not find:
        return True
    else:
        row, col = find

    for i in range(1,10):

        if valid_board(board, i, (row, col)):
            board[row][col] = i

            if solve_sudoku(board):
                return True

            board[row][col] = 0

    return False
```

```python
def valid_board(board, num, pos):
```

- This function checks if placing the given num at the specified pos (row, column) in the board is valid according to Sudoku rules.
- It performs three checks:
    - **Row Check:** Checks if the num already exists in the same row.
    - **Column Check:** Checks if the num already exists in the same column.
    - **Subgrid Check:** Checks if the num already exists within the 3x3 subgrid containing the pos.
- Returns True if the placement is valid, otherwise False.

```
def solve_sudoku(board):
```

- This function implements the backtracking algorithm to solve the Sudoku puzzle.
- **Find Empty Cell:** Calls find_next_empty_cell() to find the next empty cell in the board.
- **Base Case (No Empty Cells):** If no empty cells are found, it means the puzzle is solved, so it returns True.
- **Trying Numbers:**
  - Iterates through numbers from 1 to 9.
  - For each number, checks if it's a valid placement using valid_board().
  - If valid, places the number in the current cell.
  - Recursively calls solve_sudoku() to continue filling the rest of the grid.
  - If the recursive call returns True (meaning a solution is found), it also returns True.
  - If no solution is found with the current number, it backtracks by resetting the cell to 0.
- **No Solution:** If none of the numbers from 1 to 9 result in a valid solution, it returns False, indicating that the current branch of the search has failed.

Conclusion

This project showed us how to use AI to play games like Tic-Tac-Toe and Sudoku. We built AI players that can make smart moves and solve puzzles. We learned about how AI uses rules and strategies to win.

We also learned about how to make the AI better, like using faster search methods. This project helped us understand how AI works and how it can be used to solve problems in many different ways.

Thanks